



vodacom

Vodacom Tanzania Limited

M-Pesa Core API Broker

Interface Documentation

(Version 2.0) Draft 1

Prepared for:

Vodacom Tanzania Limited
1st Floor, Building No. 2
Mlimani City Office Park
Sam Nujoma Road
P. O. Box 2369, Dar es Salaam

Prepared by:

InfoWise Systems Ltd
Josam House Block B, Third Floor
5 CocaCola Road, Mikocheni
P.O. Box 34659, 14112 Dar es Salaam, Tanzania
Tel: +255 22 550 4832 / +255 22 270 0216
Fax: +255 22 270 0216
Cell: +255 756 283 329
Email: info@infowise.co.tz
Web: www.infowise.co.tz



January 2017

Table of Content

1. Introduction.....	3
1.1. About MPESA Broker.....	3
1.2. Document Purpose	3
2. Interface Operation Overview	3
2.1. Third Party Payment/Disbursement Initiation	3
2.2. Third-Party Transaction Notification (Collection-only)	4
2.3. Third-Party Transaction Validation (Collection-and-Service).....	4
3. Password Encryption	4
3.1. Encrypting service provider password.....	4
3.2. Encrypting Initiator Password.....	5
4. Interface Operations Details for Disbursing Funds (B2C/B2B)	6
4.1. Authentication	7
4.2. Interface URL	8
4.3. B2C Interface Specifications	9
4.4. B2B Transfer Interface Specifications	14
4.5. B2B Pay Bill Initiation Interface Specifications	19
4.6. Transaction Reversal Interface Specifications	24
4.7. Transaction Status Query Interface Specifications.....	29
5. B2C / B2B Response Codes	34
5.1. Broker Response Codes	34
5.2. MPESA Response/Result Codes	34
6. Interface for C2B/B2B Payment Validation (Collection and Service Account)	39
6.1. Interface Operations Details C2B/B2B Payment Validation	39
6.2. Interface Specification for Collection Account Payment Validation	42
7. Interface for C2B/B2B Payment Notification (Collection-only Account)	47
7.1. XML over HTTP	48
7.2. File Transfer Protocol - FTP	49
7.3. Simple Object Access Protocol - SOAP	50
Appendix	51
C2B Notification Simple Object Access Protocol - SOAP Interface Specification	51

1. Introduction

1.1. About MPESA Broker

M-Pesa Core API Broker is a system that enable third party organisations and Vodacom's internal systems to initiate transactions, receive payment notification and validation of payments done in M-Pesa platform. Core API Broker provide various integration patterns to serve different partners. In this version the MPESA Core API Broker supports XML/HTTPs only as an external payment disbursing interface XML/HTTP as well as FTP, and SOAP in notification/validation interface.

1.2. Document Purpose

This document defines interface supported by the M-Pesa Core API Broker as means of data exchange with third-party systems. Also it includes some message samples that will be produced during actual data transfer.

2. Interface Operation Overview

2.1. Third Party Payment/Disbursement Initiation

Transactions can be initiated via broker as business-to-customer (B2C) or business to business (B2B).

B2C transaction include the following possible command IDs

- BusinessPayment
- BusinessPaymentWithWithdrawalChargePaid
- SalaryPayment
- SalaryPaymentWithWithdrawChargePaid
- PromotionPayment

B2B transactions include the following command IDs

- BusinessToBusinessTransfer : This shall cover
 - o Business Buy Goods
 - o Disbursement of Business funds
 - o Merchant to Merchant
 - o Working account to Merchant account
 - o Merchant account to Working account
 - o Utility account to Working account
 - o Working account to Utility account
 - o Intra Account Transfer
 - o Inter Account Transfer

- BusinessPayBill: Business transfer to a Buy Good account

Other type of transactions

- ReverseTransaction: enabling Businesses to reverse payments done in their accounts
- QueryTransactionStatus: Enabling business to inquire the status of transactions initiated in their account

2.2. Third-Party Transaction Notification (Collection-only)

Broker is capable of receiving notification from MPESA and notify the business (third party) with the collection account when a subscriber or another business makes payment into the specific collection account.

When transaction notification are received in broker from MPESA, broker instructs MPESA to complete the transaction for third party of this category. Thereafter broker proceeds to notify the third party details of transactions received from MPESA.

2.3. Third-Party Transaction Validation (Collection-and-Service)

In addition to B2C and B2B, Broker is capable of participating in validating transactions that have already been initiated by subscriber or another business. These are business accounts that have been configured to require external validation in MPESA. These include

- Customer-to-Business (C2B): Notifications for collection account initiated by subscriber
- Business-to-Business : Notifications for collection account (Business Pay Bill and Business Buy Goods Online)

When transaction notification are received in Broker from MPESA, Broker instructs MPESA to keep transactions in authorised status. while notifying the third party the transaction details. The third party shall respond immediately indicating transactions are successfully received from Broker for further processing. When the

3. Password Encryption

3.1. Encrypting service provider password

Encrypted password is a Base64 encoded string of the sha-256 hash of the concatenation of the spId, password and timestamp. Timestamp should be in "yyyyMMddHHmmss".

Below is the sample of how to encrypt password:

Given parameters below,

spId: 30409

password: str0NgPwd

timestamp: 20150201220403

```
spPassword = UpperCase(Base64(sha-256(spId + password + timestamp)))  
spPassword = Base64(sha-256(30409str0NgPwd20150201220403))  
.toUpperCase()
```

[Example]

MZEXMDC4RURGQJGWQKE5NKQ1MZRBQUFGODBCNUI4QZNDQKNCMJHDMZIXMKU3MJNBRTNC
MUJCNZNFNTGYMJQ0NG==

3.2. Encrypting Initiator Password

For every transaction, Third party organisation has to provide initiator username and password that will be used on their behalf to initiate a transaction. Password will be encrypted with the public key from a n X509 certificate issued specifically for this purpose.

The following algorithm must be followed to encrypt that password:

- Create a block of data to be encrypted
- Write the unencrypted password value
- Encrypt the block of data created in step 2 with the public portion of the password key certificate. Use RSA algorithm with PKCS#1.5 (not OAEP) and add the result to the encrypted stream, this becomes the encrypted password to be submitted via the broker {In Java, the scheme is short-coded as "RSA/ECB/PKCS1Padding". Alternatively you may use "RSA/NONE/PKCS1Padding"}
- Convert resulting encrypted byte array into a string using base64 encoding.

[Example]

Password:

1234abcdV

Encrypted password:

PYmnB4drdaZXhHQB5BAE3ZS0TW5aa0ToQzQC7TY6sSIZ1SgwUp/uYxTXijo2JPLPT5IL
YkNUZ3iFWXHAMCw8dY6xJNpBMG0CATI0lJ6ue2Py2HhWhgPtza3nIBGwlFzGDQu10jLU
JDheja5qIV5+zAeb6bGaZ7Nu1BMT0JU9TT1gGZ0TmaNCRS3C5voxA11lotGXaQT6s7eg
P40v2aJmEk6o0WJ2vtvrQAiPNRUQDAd0+rbGRsXhECUU0yPGGp6bVnXIi5ReLg6fQRDG
OC9dGDqMarkkqTTQ7U1oJAwmn2bPj+TjMjqbjB09VVJ6PuC2oLNyEjPFYmq6ragUQTNC
JQ==

4. Interface Operations Details for Disbursing Funds (B2C/B2B)

Broker uses XML-over-HTTP interface to receive B2C transaction requests from third party systems and sending back the response and final results

When a request is posted into Broker interface:

- Third party system initiates a HTTP connection and posts the request into the broker.
- The request is received and immediately saved into broker database
- Broker initiates a SOAP web-service connection to MPESA Core API server and submits the request for further processing, while holding the third party connection
- MPESA Core API server receives the request and issues an immediate "Response" to the broker.
- Broker receives Core API response and save the initial response status against the broker record
- Broker relays the response back to the third party system immediately after saving the details.
- Broker releases the third party system connection, and the core API connection (all this happens as a synchronous processing i.e. same third party system and core API server connections are used)

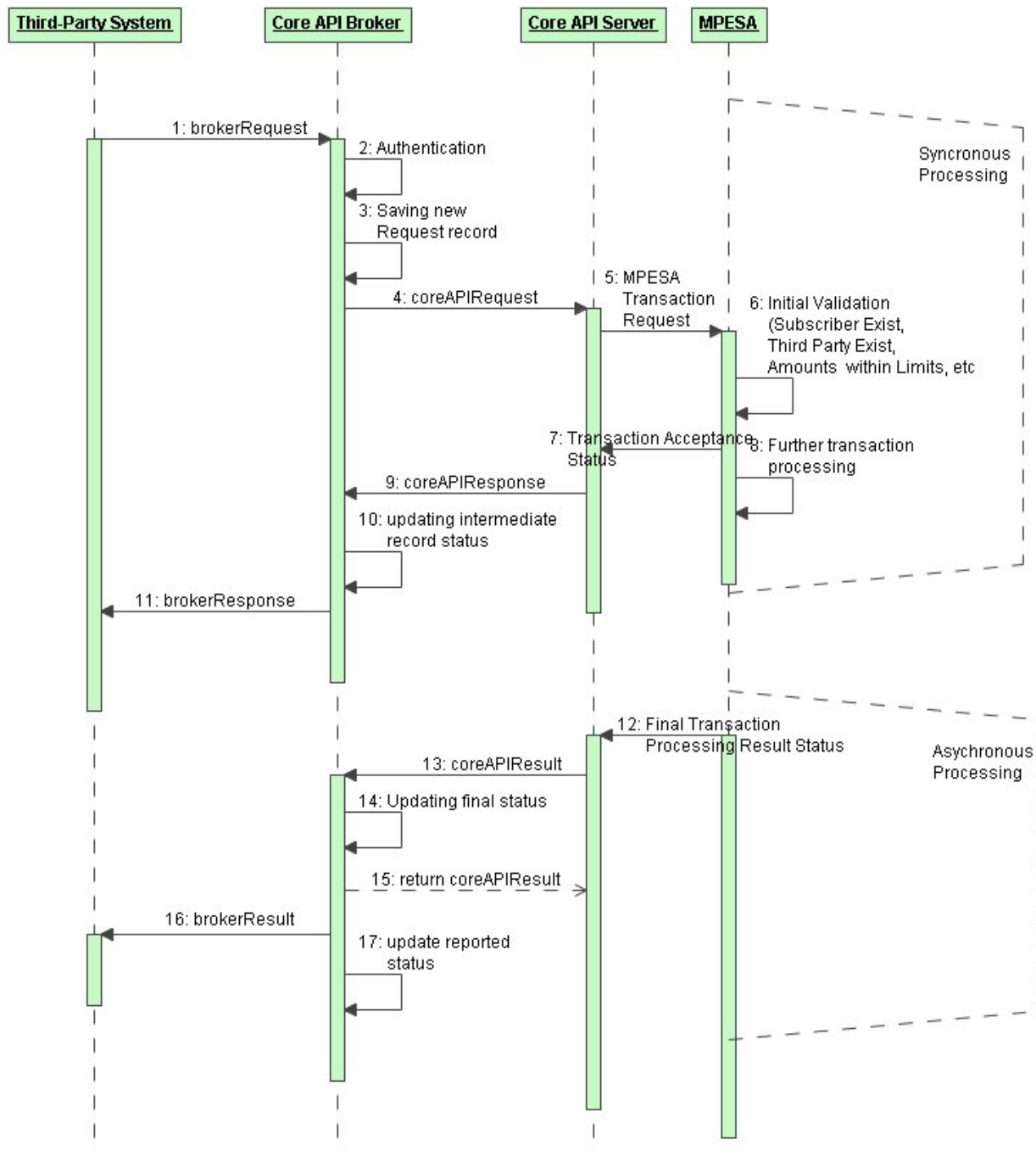
After releasing third party and core API Server connection, the broker awaits the final result from Core API server that will be submitted through a different connection from MPESA core API server when the transaction processing is completed.

When a MPESA completes the transaction processing:

- Core API server initiates a SOAP Web Service connection to the Broker and post the result
- Broker receives the result details and updates the corresponding record
- The broker acknowledges the receipt of the result back the core API server (so that the result submission is not retried by the Core API server)
- Broker establish HTTP connection to the third-party system and submits the result as HTTP-over-XML call to the third party system

- The broker updates its corresponding record to mark as successfully reported to third-party system

The following diagram summarises the interaction between Third-party system, Broker, Core API Server:



4.1. Authentication

Third party system posting requests will have to be authenticated by providing Service provider identification, "spId" and encrypted password (see [encrypting spPassword](#)), "spPassword". The

interface will only be accessible under secure HTTP (HTTPS) to ensure that data is encrypted between client and server. The Broker interface will require Mutual SSL Authentication (i.e. third party system will present to broker a Client SSL certificate to the broker and the Broker will present its server certificate during SSL connection handshake)

During third-party on-boarding process, Broker administrator will create third party's organisation account and share a password. M-Pesa business number of the organisation will be used as Service Provider Identification.

Additionally Vodacom will supply the third-party the SSL certificate that will be used to connect to the broker.

4.2. Interface URL

Broker Request should be submitted into the XML-over-HTTP interface as HTTP POST request to the interface address as:

<https://<broker-server-name-or-ip-or-address>:port/<broker-application>/b2c-request-url>

4.3. B2C Interface Specifications

4.3.1. Broker Request

The request is submitted with a format as shown below:

```
mpesaBroker version="2.0" xmlns="http://infowise.co.tz/broker/">
  <Request>
    <serviceProvider>
      <spId>900800</spId>
      <spPassword>OTcyMURGMTY5QTRENU3MUJBQkFFMEI3MkFCQzQ3RA==</spPassword>
      <timestamp>20160916153415</timestamp>
    </serviceProvider>
    <transaction>
      <commandID>BusinessPayment</commandID>
      <initiator>testinit</initiator>
      <initiatorPassword>yutoo===</initiatorPassword>
      <originatorConversationID>79-9f4-4998-9cf</originatorConversationID>
      <recipient>255765600006</recipient>
      <transactionDate>2014-09-10 10:01:45 +0300</transactionDate>
      <transactionID>10262786</transactionID>
      <requestParameters>
        <parameter>
          <key>amount</key>
          <value>200</value>
        </parameter>
      </requestParameters>
    </transaction>
  </Request>
</mpesaBroker>
```

Broker Request Attributes:

Attribute Name	Type	Optional	Description
spId	long	No	Third Party's identifier in Broker [Example] 123123
spPassword	String	No	Encrypted third-party password issued when creating account in M-Pesa broker

Timestamp	long	No	timestamp that was used to encrypt third-party password (spPassword) [Format]: yyyyMMddHHmmss [Example]: 20141027120014
commandID	String	No	Transaction type identifier [Example] BusinessPayment BusinessPaymentWithWithdrawalChargePaid SalaryPayment SalaryPaymentWithWithdrawChargePaid PromotionPayment
amount	float	No	
recipient	Long	No	Phone number of receiving party, length should be 12 [Example] 255 762 851 700
transactionDate	String	No	Timestamp of the transaction as generated by third party. [Format] yyyy-MM-dd hh:mm:ss {timezone} [Example] 2014-09-10 10:01:45 +0300
transactionID	Long	No	Third party's unique identifier of the transaction
initiator	String	No	Identifier who makes transaction on behalf of Third party. It is defined in Mpesa
initiatorPassword	String	No	Base64 string of Initiator's password encrypted (as defined at Password Encryption)
originatorConversationID	String	Yes	The unique identifier (uuid) of the request message generated by third party. It is used to identify a request between the third party and Core API. Max length

			is 128. If not specified, M-Pesa broker will generate one
--	--	--	--

4.3.2. Broker Response

When a request is sent to broker as defined in above section the Broker will respond back with response as shown below:

```
<?xml version="1.0" encoding="UTF-8"?>
<mpesaBroker version="0.2" xmlns="http://infowise.co.tz/broker/">
  <response>
    <conversationID>fd2052b5-beb3-4598-b23a-420fa1fa52ba</conversationID>
    <originatorConversationID>eeFeb8-bed763564308</originatorConversationID>
    <responseCode>0</responseCode>
    <serviceStatus>0</serviceStatus>
    <transactionID>485757</transactionID>
  </response>
</mpesaBroker>
```

Broker Response Attributes

Attribute Name	Type	Optional	Description
conversationID	String	Yes	The unique identifier generated by Mpesa for the request message.
originatorConversationID	String	No	The unique identifier generated by the broker for the request message.
responseCode	String	No	It indicates whether M-Pesa accepts the request or not.
transactionID	Long	Yes	Third party's unique identifier of the transaction
serviceStatus	String	Yes	It indicates the M-Pesa service status

4.3.3. Broker Result

Once transaction processing is done in M-Pesa Core API, Broker will send final result using third party organisation's callback URL as POST request.

The message format will be as below:

```
<?xml version="1.0" encoding="UTF-8"?>
<mpesaBroker version="0.2" xmlns="http://infowise.co.tz/broker/">
  <result>
    <resultType>Completed</resultType>
    <resultCode>0</resultCode>
    <resultDesc>Success</resultDesc>
    <originatorConversationID>eb8-be7b-43f1-a6</originatorConversationID>
    <conversationID>3ebafc33-70d8-4645-bb47-ab909efffda5</conversationID>
    <mpesaReceipt>X-AJ213</mpesaReceipt>
    <transactionID>485757</transactionID>
    <resultParameters>
      <Parameter>
        <Key>Amount</Key>
        <Value>20000</Value>
      </Parameter>
      <Parameter>
        <Key>Transaction Datetime</Key>
        <Value>2014-10-14 06:54:10</Value>
      </Parameter>
      <Parameter>
        <Key>Working Account Available Funds</Key>
        <Value>0.0000</Value>
      </Parameter>
      <Parameter>
        <Key>Utility Account Available Funds</Key>
        <Value>0.0000</Value>
      </Parameter>
      <Parameter>
        <Key>Charges Paid Account Available Funds</Key>
        <Value>0.0000</Value>
      </Parameter>
    </resultParameters>
  </result>
</mpesaBroker>
```

Broker Result Attributes;

Attribute Name	Type	Optional	Description
resultType	String	No	Indicates whether Core M-Pesa System completed processing request successful or unsuccessful. This will always be 'Completed' for B2C API

resultCode	String	No	Indicates M-Pesa completed or cancelled submitted request.
resultDesc	String	No	Description corresponding to ResultCode
originatorConversationID	String	No	Third party's unique identifier of the transaction
conversationID	String	No	
mpesaReceipt	String	Yes	Transaction receipt generated by M-Pesa
transactionID	Long	No	Third party's unique identifier of transaction
resultParameters	ParameterType	Yes	Carries specific result parameters for specific business operation

ParameterType

Parameter	Type	Description
Key	string	It indicates a parameter name.
Value	string	It indicates a parameter value.

Parameters include

Amount	Decimal	The transactional Amount
Transaction Datetime	DateTime	The date time when the transaction was completed.
Working Account Available Funds	Decimal	Amount of money in the initiating organisations M-Pesa following the transaction.
Charges Paid Account Available Funds	Decimal	
Utility Account Available Funds	Decimal	

The notified third party system has to respond to the result request with a blank response with 200 as HTTP status code (HTTP Response Code that usually appears in HTTP Header of the response).

4.4. B2B Transfer Interface Specifications

4.4.1. Broker Request

The request is submitted with a format as shown below:

```
<?xml version="1.0" encoding="UTF-8"?>
<mpesaBroker version="2.0" xmlns="http://infowise.co.tz/broker/">
  <Request>
    <serviceProvider>
      <spId>900800</spId>
      <spPassword>OTcyMURGMTY5QTRENU3MUJBQkFFMEI3MkFCQzQ3RA==</spPassword>
      <timestamp>20160916153415</timestamp>
    </serviceProvider>
    <transaction>
      <commandID>BusinessToBusinessTransfer</commandID>
      <initiator>testinit</initiator>
      <initiatorPassword>yutoo===</initiatorPassword>
      <originatorConversationID>79-9f4-4998-9cf</originatorConversationID>
      <recipient>545676</recipient>
      <transactionDate>2014-09-10 10:01:45 +0300</transactionDate>
      <transactionID>10262786</transactionID>
      <requestParameters>
        <parameter>
          <key>amount</key>
          <value>200</value>
        </parameter>
      </requestParameters>
    </transaction>
  </Request>
</mpesaBroker>
```

Broker Request Attributes:

Attribute Name	Type	Optional	Description
spId	long	No	Third Party's identifier in Broker [Example] 123123
spPassword	String	No	Encrypted third-party password issued when creating account in M-Pesa broker
timestamp	long	No	timestamp that was used to encrypt third-party password (spPassword) [Format]: yyyyMMddHHmmss

			[Example]: 20141027120014
commandID	String	No	Transaction type identifier [Example] BusinessToBusinessTransfer BusinessBuyGoods BusinessDeposit BusinessDisburseFunds
amount	float	No	
recipient	Long	No	Business number of receiving party, length should be 6 [Example] 456767
transactionDate	String	No	Timestamp of the transaction as generated by third party. [Format] yyyy-MM-dd hh:mm:ssZ{timezone} [Example] 2014-09-10 10:01:45 +0300
transactionID	Long	No	Third party's unique identifier of the transaction
initiator	String	No	Identifier who makes transaction on behalf of Third party. It is defined in Mpesa
initiatorPassword	String	No	Base64 string of Initiator's password encrypted (as defined at Password Encryption)
originatorConversationID	String	Yes	The unique identifier (uuid) of the request message generated by third party. It is used to identify a request between the third party and Core API. Max length is 128. If not specified, M-Pesa broker will generate one

4.4.2. Broker Response

When a request is sent to broker as defined in above section the Broker will respond back with response as shown below:

```
<?xml version="1.0" encoding="UTF-8"?>
<mpesaBroker version="2.0" xmlns="http://infowise.co.tz/broker/">
  <response>
    <conversationID>fd2052b5-beb3-4598-b23a-420fa1fa52ba</conversationID>
    <originatorConversationID>eeFeb8-bed763564308</originatorConversationID>
    <responseCode>0</responseCode>
    <serviceStatus>0</serviceStatus>
    <transactionID>485757</transactionID>
  </response>
</mpesaBroker>
```

Broker Response Attributes

Attribute Name	Type	Optional	Description
conversationID	String	Yes	The unique identifier generated by Mpesa for the request message.
originatorConversationID	String	No	The unique identifier generated by the broker for the request message.
responseCode	String	No	It indicates whether M-Pesa accepts the request or not.
transactionID	Long	Yes	Third party's unique identifier of the transaction
serviceStatus	String	Yes	It indicates the M-Pesa service status

4.4.3. Broker Result

Once transaction processing is done in M-Pesa Core API, Broker will send final result using third party organisation's callback URL as POST request.

The message format will be as below:

```
<?xml version="1.0" encoding="UTF-8"?>
<mpesaBroker version="2.0" xmlns="http://infowise.co.tz/broker/">
  <result>
    <resultType>Completed</resultType>
    <resultCode>0</resultCode>
    <resultDesc>Success</resultDesc>
    <originatorConversationID>eb8-be7b-43f1-a6</originatorConversationID>
    <conversationID>3ebafc33-70d8-4645-bb47-ab909efffda5</conversationID>
    <mpesaReceipt>X-AJ213</mpesaReceipt>
    <transactionID>485757</transactionID>
    <resultParameters>
      <Parameter>
        <Key>Amount</Key>
        <Value>20000</Value>
      </Parameter>
      <Parameter>
        <Key>Date</Key>
        <Value>2014-10-14 06:54:10</Value>
      </Parameter>
    </resultParameters>
  </result>
</mpesaBroker>
```

Broker Result Attributes:

Attribute Name	Type	Optional	Description
resultType	String	No	Indicates whether Core M-Pesa System completed processing request successful or unsuccessful. This will always be 'Completed' for B2C API
resultCode	String	No	Indicates M-Pesa completed or cancelled submitted request.
resultDesc	String	No	Description corresponding to ResultCode
originatorConversationID	String	No	Third party's unique identifier of the transaction
conversationID	String	No	

mpesaReceipt	String	Yes	Transaction receipt generated by M-Pesa
transactionID	Long	No	Third party's unique identifier of transaction
resultParameters	ParameterType	Yes	Carries specific result parameters for specific business operation

ParameterType

Parameter	Type	Description
Key	string	It indicates a parameter name.
Value	string	It indicates a parameter value.

Parameters include

Amount	Decimal	The transactional Amount
Transaction Datetime	DateTime	The date time when the transaction was completed.
Working Account Available Funds	Decimal	Amount of money in the initiating organisations M-Pesa following the transaction.
Charges Paid Account Available Funds	Decimal	
Utility Account Available Funds	Decimal	

The notified third party system has to respond to the result request with a blank response with 200 as HTTP status code (HTTP Response Code that usually appears in HTTP Header of the response).

4.5. B2B Pay Bill Initiation Interface Specifications

4.5.1. Broker Request

The request is submitted with a format as shown below:

```
<?xml version="1.0" encoding="UTF-8"?>
<mpesaBroker version="2.0" xmlns="http://infowise.co.tz/broker/">
  <Request>
    <serviceProvider>
      <spId>900800</spId>
      <spPassword>OTcyMURGCMEE1QzJEMzU3MUQkFFMEI3MkFCQzQ3RA==</spPassword>
      <timestamp>20160916153415</timestamp>
    </serviceProvider>
    <transaction>
      <commandID>BusinessPayBill</commandID>
      <initiator>testinit</initiator>
      <initiatorPassword>yutoo===</initiatorPassword>
      <originatorConversationID>ddk9-9ff4-498-f</originatorConversationID>
      <recipient>677887</recipient>
      <transactionDate>2014-09-10 10:01:45 +0300</transactionDate>
      <transactionID>10262786</transactionID>
      <requestParameters>
        <parameter>
          <key>reference</key>
          <value>243434343434</value>
        </parameter>
        <parameter>
          <key>amount</key>
          <value>200</value>
        </parameter>
      </requestParameters>
    </transaction>
  </Request>
</mpesaBroker>
```

Broker Request Attributes:

Attribute Name	Type	Optional	Description
spId	long	No	Third Party's identifier in Broker [Example] 123123
spPassword	String	No	Encrypted third-party password issued when creating account in M-Pesa broker
timestamp	long	No	timestamp that was used to encrypt third-party password (spPassword)

			[Format]: yyyyMMddHHmmss [Example]: 20141027120014
commandID	String	No	Transaction type identifier [Example] BusinessPayBill
amount	float	No	
recipient	Long	No	Business number of receiving party, length should be 12 [Example] 851700
transactionDate	String	No	Timestamp of the transaction as generated by third party. [Format] yyyy-MM-dd hh:mm:ss {timezone} [Example] 2014-09-10 10:01:45 +0300
transactionID	Long	No	Third party's unique identifier of the transaction
initiator	String	No	Identifier who makes transaction on behalf of Third party. It is defined in Mpesa
initiatorPassword	String	No	Base64 string of Initiator's password encrypted (as defined at Password Encryption)
originatorConversationID	String	Yes	The unique identifier (uuid) of the request message generated by third party. It is used to identify a request between the third party and Core API. Max length is 128. If not specified, M-Pesa broker will generate one

4.5.2. Broker Response

When a request is sent to broker as defined in above section the Broker will respond back with response as shown below:

```
<?xml version="1.0" encoding="UTF-8"?>
<mpesaBroker version="2.0" xmlns="http://infowise.co.tz/broker/">
  <response>
    <conversationID>fd2052b5-beb3-4598-b23a-420fa1fa52ba</conversationID>
    <originatorConversationID>eefeb8-bed763564308</originatorConversationID>
    <responseCode>0</responseCode>
    <serviceStatus>0</serviceStatus>
    <transactionID>485757</transactionID>
  </response>
</mpesaBroker>
```

Broker Response Attributes

Attribute Name	Type	Optional	Description
conversationID	String	Yes	The unique identifier generated by Mpesa for the request message.
originatorConversationID	String	No	The unique identifier generated by the broker for the request message.
ResponseCode	String	No	It indicates whether M-Pesa accepts the request or not.
TransactionID	Long	Yes	Third party's unique identifier of the transaction
ServiceStatus	String	Yes	It indicates the M-Pesa service status

4.5.3. Broker Result

Once transaction processing is done in M-Pesa Core API, Broker will send final result using third party organisation's callback URL as POST request.

The message format will be as below:

```
<?xml version="1.0" encoding="UTF-8"?>
<mpesaBroker version="2.0" xmlns="http://infowise.co.tz/broker/">
  <result>
    <resultType>Completed</resultType>
    <resultCode>0</resultCode>
    <resultDesc>Success</resultDesc>
    <originatorConversationID>eb8-be7b-43f1-a6</originatorConversationID>
    <conversationID>3ebafc33-70d8-4645-bb47-ab909efffda5</conversationID>
    <mpesaReceipt>X-AJ213</mpesaReceipt>
    <transactionID>485757</transactionID>
    <resultParameters>
      <Parameter>
        <Key>Amount</Key>
        <Value>20000</Value>
      </Parameter>
      <Parameter>
        <Key>Date</Key>
        <Value>2014-10-14 06:54:10</Value>
      </Parameter>
    </resultParameters>
  </result>
</mpesaBroker>
```

Broker Result Attributes;

Attribute Name	Type	Optional	Description
ResultType	String	No	Indicates whether Core M-Pesa System completed processing request successful or unsuccessful. This will always be 'Completed' for B2C API
ResultCode	String	No	Indicates M-Pesa completed or cancelled submitted request.
ResultDesc	String	No	Description corresponding to ResultCode
OriginatorConversationID	String	No	Third party's unique identifier of the transaction
ConversationID	String	No	

MpesaReceipt	String	Yes	Transaction receipt generated by M-Pesa
TransactionID	Long	No	Third party's unique identifier of transaction
ResultParameters	ParameterType	Yes	Carries specific result parameters for specific business operation

ParameterType

Parameter	Type	Description
Key	string	It indicates a parameter name.
Value	string	It indicates a parameter value.

Parameters include

Amount	Decimal	The transactional Amount
Date	DateTime	The date time when the transaction was completed.

The notified third party system has to respond to the result request with a blank response with 200 as HTTP status code (HTTP Response Code that usually appears in HTTP Header of the response).

4.6. Transaction Reversal Interface Specifications

4.6.1. Broker Request

The request is submitted with a format as shown below:

```
<?xml version="1.0" encoding="UTF-8"?>
<mpesaBroker version="2.0" xmlns="http://infowise.co.tz/broker/">
  <Request>
    <serviceProvider>
      <spId>900800</spId>
      <spPassword>OTcyMURGMTY5NEFCMEE1QzJEMzU3MUJBQkFQzQ3RA==</spPassword>
      <timestamp>20160916153415</timestamp>
    </serviceProvider>
    <transaction>
      <commandID>ReverseTransaction</commandID>
      <initiator>testinit</initiator>
      <initiatorPassword>yutoo===</initiatorPassword>
      <originatorConversationID>d02dke79-9ff4</originatorConversationID>
      <recipient>255765600006</recipient>
      <transactionDate>2017-09-28 17:20:52</transactionDate>
      <transactionID>10262786</transactionID>
      <requestParameters>
        <parameter>
          <key>originalMpesaReceipt</key>
          <value>3HJ76M2MHL</value>
        </parameter>
      </requestParameters>
    </transaction>
  </Request>
</mpesaBroker>
```

Broker Request Attributes:

Attribute Name	Type	Optional	Description
spId	Long	No	Third Party's identifier in Broker [Example] 123123
spPassword	String	No	Encrypted third-party password issued when creating account in M-Pesa broker
timestamp	Long	No	timestamp that was used to encrypt third-party password (spPassword) [Format]: yyyyMMddHHmmss

			[Example]: 20141027120014
commandID	String	No	Transaction type identifier [Example] ReverseTransaction
Amount	float	No	
recipient	Long	No	Phone number of receiving party, length should be 12 [Example] 255 762 851 700 (B2C or C2B) or 123456 (B2B)
transactionDate	String	No	Timestamp of the transaction as generated by third party. [Format] yyyy-MM-dd hh:mm:ss {timezone} [Example] 2014-09-10 10:01:45 +0300
transactionID	Long	No	Third party's unique identifier of the transaction
initiator	String	No	Identifier who makes transaction on behalf of Third party. It is defined in Mpesa
initiatorPassword	String	No	Base64 string of Initiator's password encrypted (as defined at Password Encryption)
initiator	String	No	Identifier who makes transaction on behalf of Third party. It is defined in Mpesa
originalMpesaReceipt	String	No	Transaction receipt of the transaction to be reversed (generated by M-Pesa)

4.6.2. Broker Response

When a request is sent to broker as defined in above section the Broker will respond back with response as shown below:

```
<?xml version="1.0" encoding="UTF-8"?>
<mpesaBroker version="2.0" xmlns="http://infowise.co.tz/broker/">
  <response>
    <conversationID>fd2052b5-beb3-4598-b23a-420fa1fa52ba</conversationID>
    <originatorConversationID>eeFeb8-bed763564308</originatorConversationID>
    <responseCode>0</responseCode>
    <serviceStatus>0</serviceStatus>
    <transactionID>485757</transactionID>
  </response>
</mpesaBroker>
```

Broker Response Attributes

Attribute Name	Type	Optional	Description
conversationID	String	Yes	The unique identifier generated by Mpesa for the request message.
originatorConversationID	String	No	The unique identifier generated by the third broker for the request message.
responseCode	String	No	It indicates whether M-Pesa accepts the request or not.
transactionID	Long	Yes	Third party's unique identifier of the transaction
serviceStatus	String	Yes	It indicates the M-Pesa service status

4.6.3. Broker Result

Once transaction processing is done in M-Pesa Core API, Broker will send final result using third party organisation's callback URL as POST request.

The message format will be as below:

```
<?xml version="1.0" encoding="UTF-8"?>
<mpesaBroker version="2.0" xmlns="http://infowise.co.tz/broker/">
  <result>
    <resultType>Completed</resultType>
    <resultCode>0</resultCode>
    <resultDesc>Success</resultDesc>
    <originatorConversationID>eb8-be7b-43f1-a6</originatorConversationID>
    <conversationID>3ebafc33-70d8-4645-bb47-ab909effffda5</conversationID>
    <mpesaReceipt>X-AJ213</mpesaReceipt>
    <transactionID>485757</transactionID>
    <resultParameters>
      <Parameter>
        <Key>Amount</Key>
        <Value>20000</Value>
      </Parameter>
      <Parameter>
        <Key>Date</Key>
        <Value>2014-10-14 06:54:10</Value>
      </Parameter>
    </resultParameters>
  </result>
</mpesaBroker>
```

Broker Result Attributes;

Attribute Name	Type	Optional	Description
resultType	String	No	Indicates whether Core M-Pesa System completed processing request successful or unsuccessful. This will always be 'Completed' for B2C API
resultCode	String	No	Indicates M-Pesa completed or cancelled submitted request.

resultDesc	String	No	Description corresponding to ResultCode
originatorConversationID	String	No	Third party's unique identifier of the transaction
conversationID	String	No	
mpesaReceipt	String	Yes	Transaction receipt generated by M-Pesa
transactionID	Long	No	Third party's unique identifier of transaction
resultParameters	ParameterType	Yes	Carries specific result parameters for specific business operation

ParameterType

Parameter	Type	Description
Key	string	It indicates a parameter name.
Value	string	It indicates a parameter value.

Parameters include

Amount	Decimal	The transactional Amount
Date	DateTime	The date time when the transaction was completed.

The notified third party system has to respond to the result request with a blank response with 200 as HTTP status code (HTTP Response Code that usually appears in HTTP Header of the response).

4.7. Transaction Status Query Interface Specifications

4.7.1. Broker Request

The request is submitted with a format as shown below:

```
<?xml version="1.0" encoding="UTF-8"?>
<mpesaBroker version="2.0" xmlns="http://infowise.co.tz/broker/">
  <Request>
    <serviceProvider>
      <spId>900800</spId>
      <spPassword>ORENDNEFCMEE1QzJEMzU3MUJBQkFFMEI3MkFC3RA==</spPassword>
      <timestamp>20160916153415</timestamp>
    </serviceProvider>
    <transaction>
      <commandID>QueryTransactionStatus</commandID>
      <initiator>testinit</initiator>
      <initiatorPassword>yutoo===</initiatorPassword>
      <originatorConversationID>d0279-9ff4-499f</originatorConversationID>
      <recipient>255765600006</recipient>
      <transactionDate>2014-09-10 10:01:45Z+0300</transactionDate>
      <transactionID>10262786</transactionID>
    </transaction>
  </Request>
</mpesaBroker>
```

Broker Request Attributes:

Attribute Name	Type	Optional	Description
spId	Long	No	Third Party's identifier in Broker [Example] 123123
spPassword	String	No	Encrypted third-party password issued when creating account in M-Pesa broker
timestamp	Long	No	timestamp that was used to encrypt third-party password (spPassword) [Format]: yyyyMMddHHmmss [Example]: 20141027120014
commandID	String	No	Transaction type identifier [Example] QueryTransactionStatus

amount	Float	No	
recipient	Long	No	Phone number of receiving party, length should be 12 [Example] 255 762 851 700
transactionDate	String	No	Timestamp of the transaction as generated by third party. [Format] yyyy-MM-dd hh:mm:ssZ{timezone} [Example] 2014-09-10 10:01:45Z+0300
transactionID	Long	No	Third party's unique identifier of the transaction
Initiator	String	No	Identifier who makes transaction on behalf of Third party. It is defined in Mpesa
initiatorPassword	String	No	Base64 string of Initiator's password encrypted (as defined at Password Encryption)
originatorConversationID	String	Yes	The unique identifier (uuid) of the request message generated by third party. It is used to identify a request between the third party and Core API. Max length is 128. If not specified, M-Pesa broker will generate one

4.7.2. Broker Response

When a request is sent to broker as defined in above section the Broker will respond back with response as shown below:

```
<?xml version="1.0" encoding="UTF-8"?>
<mpesaBroker version="2.0" xmlns="http://infowise.co.tz/broker/">
  <response>
    <conversationID>fd2052b5-beb3-4598-b23a-420fa1fa52ba</conversationID>
    <originatorConversationID>eeFeb8-bed763564308</originatorConversationID>
    <responseCode>0</responseCode>
    <serviceStatus>0</serviceStatus>
    <transactionID>485757</transactionID>
  </response>
</mpesaBroker>
```

Broker Response Attributes

Attribute Name	Type	Optional	Description
conversationID	String	Yes	The unique identifier generated by Mpesa for the request message.
originatorConversationID	String	No	The unique identifier generated by the broker for the request message.
responseCode	String	No	It indicates whether M-Pesa accepts the request or not.
transactionID	Long	Yes	Third party's unique identifier of the transaction
serviceStatus	String	Yes	It indicates the M-Pesa service status

4.7.3. Broker Result

Once transaction processing is done in M-Pesa Core API, Broker will send final result using third party organisation's callback URL as POST request.

The message format will be as below:

```
<?xml version="1.0" encoding="UTF-8"?>
<mpesaBroker version="2.0" xmlns="http://infowise.co.tz/broker/">
  <result>
    <resultType>Completed</resultType>
    <resultCode>0</resultCode>
    <resultDesc>Success</resultDesc>
    <originatorConversationID>eb8-be7b-43f1-a6</originatorConversationID>
    <conversationID>3ebafc33-70d8-4645-bb47-ab909efffda5</conversationID>
    <mpesaReceipt>X-AJ213</mpesaReceipt>
    <transactionID>485757</transactionID>
    <resultParameters>
      <Parameter>
        <Key>Amount</Key>
        <Value>20000</Value>
      </Parameter>
      <Parameter>
        <Key>Date</Key>
        <Value>2014-10-14 06:54:10</Value>
      </Parameter>
    </resultParameters>
  </result>
</mpesaBroker>
```

Broker Result Attributes;

Attribute Name	Type	Optional	Description
resultType	String	No	Indicates whether Core M-Pesa System completed processing request successful or unsuccessful. This will always be 'Completed' for B2C API
resultCode	String	No	Indicates M-Pesa completed or cancelled submitted request.

resultDesc	String	No	Description corresponding to ResultCode
originatorConversationID	String	No	Third party's unique identifier of the transaction
conversationID	String	No	
mpesaReceipt	String	Yes	Transaction receipt generated by M-Pesa
transactionID	Long	No	Third party's unique identifier of transaction
resultParameters	ParameterType	Yes	Carries specific result parameters for specific business operation

ParameterType

Parameter	Type	Description
Key	String	It indicates a parameter name.
Value	String	It indicates a parameter value.

Parameters include

Amount	Decimal	The transactional Amount
Date	DateTime	The date time when the transaction was completed.

The notified third party system has to respond to the result request with a blank response with 200 as HTTP status code (HTTP Response Code that usually appears in HTTP Header of the response).

5. B2C / B2B Response Codes

5.1. Broker Response Codes

19 `ApiRequestMessageExpiryFailure`

If the gap between request timestamp and time at which request is received by transaction processor is too big, request will be rejected.

1007 `AuthenticationFailure`

Third party organisation's credentials cannot be validated by M-Pesa broker

1008 `DuplicateDetected`

Duplicate transaction request has been submitted to the Broker (the same `spld` and `transactionID`)

5.2. MPESA Response/Result Codes

0 Success

This is indicated for both Response and Result messages. In Response value of 0 indicates that the request has passed basic validation tests and been passed on to the core system for further processing. An Result value of 0 means that the requested financial transaction has been completed successfully. In case of multi staged transaction the response code will be 43 for initiation stage and 46 for confirmation.

1 InsufficientFunds

Typically indicates that the primary party does not have enough money to complete the requested financial transaction. In the case of the B2C API, this means that the business organisation does not have enough money in its utility account.

This is returned by the *Result* and represented internally as a transaction reason.

2 LessThanMinimumTransactionValue

3 MoreThanMaximumTransactionValue

Each of these potential failures derive from the rules which govern MPesa financial transactions. In the case of the B2C API, these are taken from *DefaultBusinessUtilityAccount* rules, which specify a lower bound of 10 KE shillings and an upper bound of 70000. For promotion payments and salary payments to unregistered users, the upper limit is lower: 35000. Likewise, the lower bound for these 2 transactions is also different: 101 rather than 10.

These are both returned by the *Result* and represented internally as a transaction reason.

4 **WouldExceedDailyTransferLimit**

This is a limit on daily activity. For the business organisation, this is very high: XXXXX TZ shillings. It is much more likely that this rule will apply to the customer, where the limit is XXXXX TZ shillings.

This is returned by the *Result* and represented internally as a transaction reason.

5 **WouldExceedMinimumBalance**

This rule is rather confusingly named – it actually means that a transaction would bring the business organisations utility account below the required minimum – which is currently 0.

This is returned by the *Result* and represented internally as a transaction reason. It is computed during execution of the financial transaction.

6 **UnresolvedPrimaryParty**

7 **UnresolvedReceiverParty**

The primary party or receiver party cannot be associated with an MPesa identity. For transactions (such as promotion payment and salary payment) which are supported for unregistered users, the unresolved receiver party rule is not applied.

These are returned in the *Result* and checked before the linked financial transaction is created.

8 **WouldExceedMaximumBalance**

For the B2C API, this applies to the recipient MMF customer – the limit here is 100,000 KE shillings.

This is returned by the *Result* and represented internally as a transaction reason. It is computed during execution of the financial transaction.

9 **LessThanMinAirtimeValue**

10 **MoreThanMaxAirtimeValue**

Neither of these apply to B2C API operations.

11 **DebitAccountInvalid**

12 **CreditAccountInvalid**

13 **UnresolvedDebitAccount**

14 **UnresolvedCreditAccount**

These are all existing failure transaction reasons for B2C financial transactions – they would be issued to the caller in the *Result* message. The latter 2 in particular are

unlikely to occur, as the request would be rejected by Core API specific checks before the request reached the accounting engine.

15 DuplicateDetected

~~Currently, any requests which presents an originator conversation id which has been seen before will be rejected. This rule will need to be reconsidered in contexts which require multi-stage conversations or multiple Callers. This check (along with the message expiry check) is the first thing done by the transaction processor when handling an API request. The outcome is reported in the *Result* message. (OriginatorConversationID has been seen before in M-Pesa Core API, transaction will be rejected)~~

16 PayUtilityInvalidAccountNumberFormat

Does not apply to the B2C API operations.

17 InternalFailure

A catch all for failures which are not identified more specifically – this can occur in either the *Response* or *Result* – although the intent is to replace any such error with a more precise message.

18 InitiatorCredentialCheckFailure

The password check for the initiator failed, either because the presented password is wrong, or something has gone wrong in the encryption or decryption steps. This is issued in the *Result* before the creation of a financial transaction.

19 ApiRequestMessageExpiryFailure

Initiators have the option to specify a request timestamp. If the gap between this value and the time at which the message is received by the transaction processor is too great, then the request is rejected. The specific interval is configured separately for each API operation (and indeed, for each API operation stage). This is the first check performed on an API request arriving at the transaction processor, and as such is included in the *Result* message.

20 UnresolvedInitiator

The initiator username presented with the request cannot be found. This is included in the *Result*.

21 InitiatorToPrimaryPartyPermissionFailure

The initiator presented does not have the right to issue requests for the specified primary party (as established during creation of the initiator on the admin web site). The is part of the *Result*, and computed prior to issuing a financial transaction.

22 InitiatorStatusCheckFailure

The presented initiator username can be received, but the initiator is not currently active. This fact is returned in the *Result*.

23 RequestSchemaValidationError

Incoming API requests are validated against the schema defined in *CPSInterface_Request.xsd*. This is part of the *ApiResponse* message – the details of the validation error are included in the response.

24 MissingRequestParameters

Required input parameters are defined for each type of API operation. If these are missing, then this is issued in the *Result* message. The names of the missing parameters are included in the result parameters field.

25 InvalidRequestParameters

If all required parameters are presented, then validation checks are performed. Specifically, the parameter is checked to see if it can be converted to the intended type, and then checked against (optional) configuration regular expression based validation rules. For the B2C API, only the type check (i.e the amount specified is a valid decimal) is performed. Failures are indicated in the *Result* message, with the details included in the result parameters.

26 SystemTooBusy

Included in the *ApiResponse* message if a traffic blocking condition is in place. See section 5.1 for more detail.

28 InitiatorAllowedOperationCheckFailure

Upon creation, initiators are assigned permissions for specific API operations – if the operation specified in the request message is not included in this list, then this *Result* message is received.

29 InvalidCommand

The command specified in the request is not defined – this is part of the *Result*.

30 ErrorSerializingRequest

After XML schema validation, the API attempts to convert the XML request into an internal *ApiRequest* object – any failures result in this *ApiResponse* message. The details of the exception are included.

31 InitiatorNotSpecified

The initiator username is not specified, or not specified in a way that can be parsed.

32 ErrorSerializingRequest

The presented identifier for the initiator is not a username.

33 PrimaryPartyNotSpecified

34 PrimaryPartyIdentifierInvalid

35 ReceiverPartyNotSpecified

36 ReceiverPartyIdentifierInvalid

In all these case, one of the API parties is either missing from the request, or has been presented with the wrong identifier type., This kind of failure is indicated in the *Result* message.,

37 MissingApiCommand

No command is included in the request – in general this problem should be captured by the XML schema validation, but if it is not this *Result* message is issued.

38 InvalidConversationId

39 UnknownConversationId

Not currently checked for – this will take place for multi-stage conversations.

40 InvalidParameterDefinition

Not been used.

41 DuplicateConversationDetected

If the originatorconversationid is be reused again, the request will end up with duplicate error code. The request should have a unique originatorconversationid for every new apirequest.

42 DuplicateStageMessageDetected

If the stage name is requested twice it will end up with duplicate stage. Though the occurrence of this would be very rare.

43 AwaitingConfirmation

It is applicable for multi stage transaction. Once the first stage of the transaction is set to authorised with will set the result code as AwaitingConfirmation to convey the initiator to initiate the confirmation stage.

44 InitiatorToReceiverPartyPermissionFailure

The initiator presented does not have the right to issue requests for the specified receiver party (as established during creation of the initiator on the admin web site). The is part of the *Result*, and computed prior to issuing a financial transaction.

45 InternalErrorDuringFinancialTransaction

46 ConfirmationReceived

It is applicable for the multi stage transaction. The result code is send once the confirmation stage is processed successfully to confirm the financial transaction.

47 RejectionReceived

It is applicable for the multi stage transaction. The result code is send once the confirmation stage is processed successfully to cancel the financial transaction

48 OperationPermissionFailure

For transaction status query if the initiator presented was not the part of the original transaction or linked to the respective parties within the original request a transaction access permission failure result is send out. For balance query if the initiator is not linked to the request organisation shortcode it will return the same result code. As the validation is common the result code will be shared by both the request.

49 NoTransactionFound

For transaction status query is the request transaction parameter is not found in the system it will send the transaction not found result code.

50 InitiatorOrgIdentityStatusFailure

The result code is returned if the organisation status is closed.

51 DebitChargeAccountInvalid

If the Airtime purchase account is set to inactive state the corresponding transaction will return an invalid debit charge account failure.

52 WouldExceedMaximumSingleAirtimePurchase

If transaction amount exceeds the maximum amount configured for single request it will fail the transaction for rule failure.

53 MerchantTransferWithinHierarchyNotAllowed

The merchant to merchant transaction is not allowed within the same hierarchy. In case its attempted a prerequired action associated to the transaction will fail and return failure response code.

54 AccountBalanceBelowMinimumThresholdFailure

The exception response code will be raised if the float balance of the organisation goes below the threshold value configured for the respective rulegroup for agency redistribution of funds transaction type.

55 InvalidRequestToExecuteTransactionsOnSingleAccount

The error code will be generated in case same party is used as primary and receiver party for transactions like business to business transaction were the primary party should be different to receiver party.

56 CheckSumFailure

In case the password is tampered the same will fail at the validation on coreapi web service and it will return the negative result code as api response.

6. Interface for C2B/B2B Payment Validation (Collection and Service Account)

When C2B or B2B payment is made in MPESA into a business collection account, the broker will receive the request from USSD Menu and forward into Third parties systems.

After the broker forward the payment information to third party via its HTTP/XML interface.

The third party is expected to immediately receive, extract and save the request details and respond to the broker acknowledging receiving the request with the unique serviceID

Once the service provider completes the processing the request, the service provider system shall return to broker their final processing result indicating whether the transaction shall be finalized or cancelled in MPESA. When the service has been provided to the respective account specified by accountReference, the service provider system sends final results details to the broker providing receipts issued by service provider (serviceReceipt) and the date the service was provisioned by the service provider system (serviceDate)

Third party must implement a HTTP over XML request/response interface that supports the broker interface.

6.1. Interface Operations Details C2B/B2B Payment Validation

Broker uses XML-over-HTTP interface to receive C2B transaction requests from M-pesa,

- Customer initiate transaction via USSD to mpesa
- M-pesa initiates a SOAP web-service connection to broker server and submits the request for further processing, while holding the release Customer connection from USSD
- Broker server receives the request and issues and immediate "Response" to the Mpesa core api.

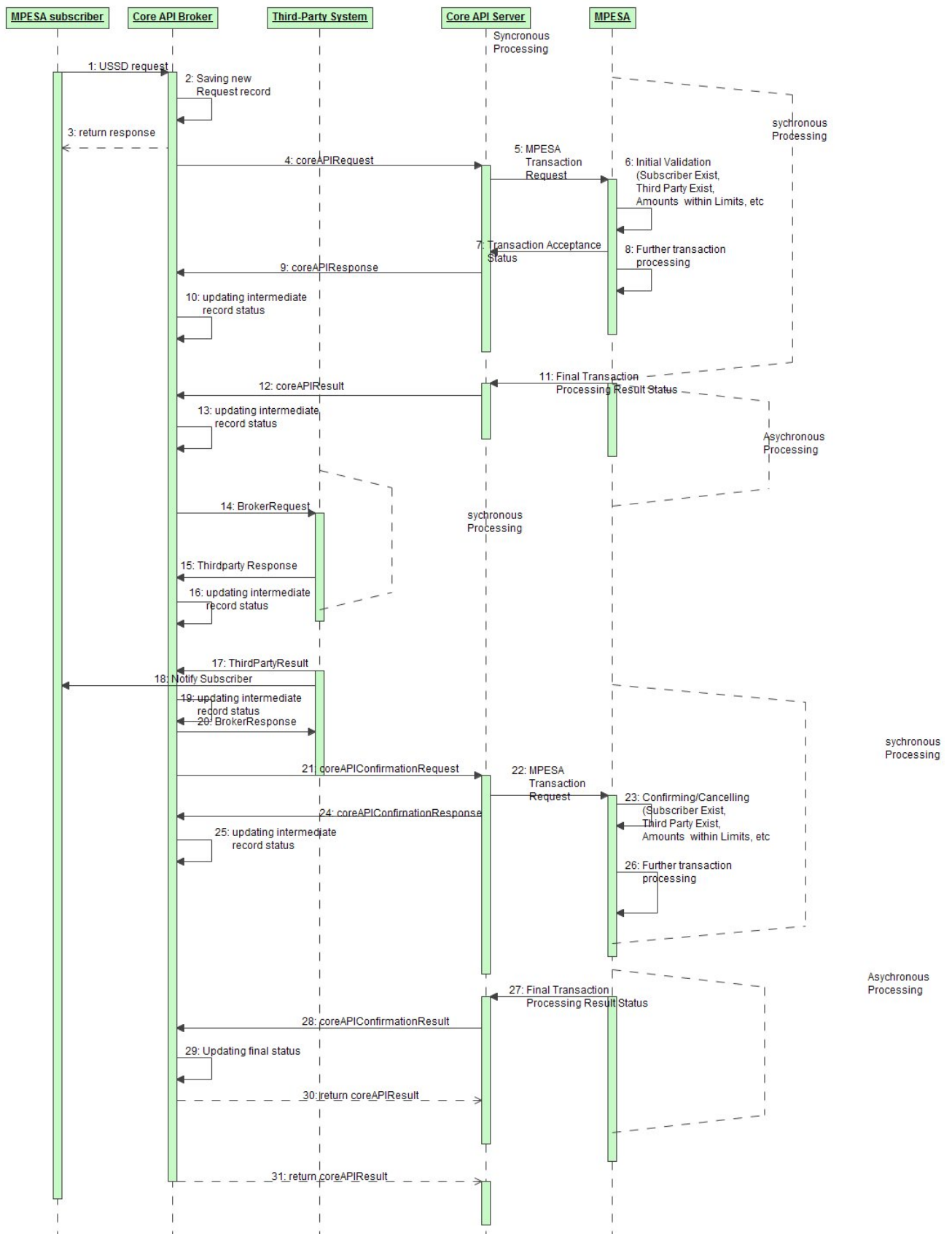
When a Broker save transaction received from M-pesa:

- Broker establish HTTP connection to the third-party system and submits the result as HTTP-over-XML call to the third party system
- Third-party server receives the request and issues and immediate "Response" to the broker.
- Broker receives the response details and updates the corresponding record

When a Third-party completes the transaction processing:

- Third-party establish HTTP connection to the third-party system and submits the result as HTTP-over-XML call to the broker system
- Broker acknowledge third-party received result successfully
- Broker initiates a SOAP web-service connection to MPESA Core API server and submits the request for further processing
- MPESA Core API server receives the request and issues and immediate "Response" to the broker.
- Broker receives Core API response and save the initial response status against the broker record i.e. Confirming/Cancelling
- When MPESA Core API completes transaction processing, Core API server initiates a SOAP Web Service connection to the Broker and post the result

The following diagram summarises the interaction between Broker, Core API Server and Third-party system:



6.2. Interface Specification for Collection Account Payment Validation

6.2.1. Broker Request

Broker Request sent to third-party should be submitted into the XML-over-HTTP interface as HTTP POST request to the interface address as:

<https://<third-party-name-or-ip-or-address>:port/<third-party-application>/C2B-request-url>

The request is submitted with a format as shown below:

```
<?xml version="1.0" encoding="UTF-8"?>
<mpesaBroker xmlns="http://infowise.co.tz/broker/" version="2.0">
  <request>
    <serviceProvider>
      <spId>999999</spId>
      <spPassword>encryptedPassword0</spPassword>
      <timestamp>20141026092954</timestamp>
    </serviceProvider>
    <transaction>
      <amount>500.0</amount>
      <commandID>Pay Bill</commandID>
      <initiator>255762851700</initiator>
      <originatorConversationID>eef8deb8-be7b-43</originatorConversationID>
      <recipient>999999</recipient>
      <mpesaReceipt>X-AJ213</mpesaReceipt>
      <transactionDate>2014-10-26 09:29:54</transactionDate>
      <transactionID>1094</transactionID>
      <conversationID>23d13181-ca90-42c6-b29c-196e45519277</conversationID>
      <resultURL>http://brokerip:port/app/url</resultURL>
    </transaction>
  </request>
</mpesaBroker>
```

Request Attribute Definition

Attribute Name	Type	Optional	Description
spId	long	No	Third Party's (service provider) identifier in Broker. This is 6 digit C2B business number [Example] 999999
spPassword	String	No	Encrypted third-party password issued when creating account in M-Pesa broker The password must be Base64 encoded string of the SHA-256 hash of concatenation of spId, password and timestamp as described in section ??? Encrypting spPassword
Timestamp	long	No	timestamp that was used to encrypt third-party password (spPassword) [Format]: yyyyMMddHHmmss [Example]: 20141027120014
Amount	float	No	
commandID	String	No	Transaction type identifier [Example] PayBill BuyGoodsOnline CustomerBuyGoodsOnline BundlePurchase
Recipient	Long	No	Short code of the service provider (receiving party), length should be 6 digits [Example] 123000
transactionDate	String	No	Timestamp of the transaction as generated by third party. [Format] yyyy-MM-dd HH:mm:ssZ [Example] 2014-10-26 09:29:54+0300

accountReference	String	No	Account number in the third-party system that shall payment shall be made against and service (e.g. bank account, metre number, etc.)
Initiator	String	No	Phone number of subscriber who initiate transaction in MPESA
initiatorPassword	String	No	Base64 string of Initiator's password encrypted as defined in section ??? (Initiator Password Encryption)
originatorConversationID	String	Yes	The unique identifier of the request message generated by broker. It is used to identify a request between the broker and Core API.

Third-party will respond back with "Response" as shown below:

```
<?xml version="1.0" encoding="UTF-8"?>
<mpesaBroker xmlns="http://infowise.co.tz/broker/" version="2.0">
  <response>
    <conversationID>fd2052b5-beb3-4598-b23a-420fa1fa52ba</conversationID>
    <originatorConversationID>eef8deb8-bed763b7564308</originatorConversationID>
    <responseCode>0</responseCode>
    <responseDesc>Received</responseDesc>
    <serviceStatus>Success</serviceStatus>
    <serviceID>24182402</serviceID>
    <transactionID>485757</transactionID>
  </response>
</mpesaBroker>
```

Response Attribute Definition

Attribute Name	Type	Optional	Description
conversationID	String	Yes	The unique identifier generated by Mpesa for the payment.
originatorConversationID	String	No	The unique identifier of the payment generated by the broker.
transactionID	Long	Yes	Broker's serial unique identifier of the transaction

serviceID	Long	Yes	Third party's unique identifier of the transaction
responseCode	String	No	It indicates whether M-Pesa accepts the request or not.
serviceStatus	String	Yes	It indicates the M-Pesa service status

6.2.2. Third-Party Result

Once transaction processing is done in M-Pesa Core API, Broker will send final result

```
<?xml version="1.0" encoding="UTF-8"?>
<mpesaBroker xmlns="http://infowise.co.tz/broker/" version="2.0">
  <result>
    <serviceProvider>
      <spId>999999</spId>
      <spPassword>encryptedPassword0</spPassword>
      <timestamp>20141026092954</timestamp>
    </serviceProvider>
    <transaction>
      <resultType>Completed</resultType>
      <resultCode>0</resultCode>
      <resultDesc>Success</resultDesc>
      <serviceReceipt>123456-2344</serviceDate>
      <serviceDate>2014-10-26 09:29:54+0300</serviceDate>
      <originatorConversationID>eef8deb86</originatorConversationID>
      <conversationID>3ebafc33-70d8-4645-bb47-ab909efffda5</conversationID>
      <transactionID>485757</transactionID>
      <initiator>999999</initiator>
      <initiatorPassword>encryptedPassword1</initiatorPassword>
    </transaction>
  </result>
</mpesaBroker>
```

Third Party Result Attribute Definition

Attribute Name	Type	Optional	Description
spId	long	No	Third Party's identifier in Broker [Example] 999999
spPassword	String	No	Encrypted third-party password issued when creating account in M-Pesa broker The password must be Base64 encoded string of the SHA-256 hash of concatenation of spId, password and timestamp Generating spPassword spPassword = Base64(SHA-256(spId+password+timestamp)) [Example] spId: 123123 password: str0ngP@ssw0rd timestamp: 20141027120014 spPassword=Base64(SHA-256(123123str0ngP@ssw0rd20141027120014))
Timestamp	long	No	timestamp that was used to encrypt third-party password (spPassword) [Format]: yyyyMMddHHmmss [Example]: 20141027120014
Amount	float	No	
resultType	String	No	'Completed' when processing is completed
resultCode	Long	No	'0' when service has been successfully provisioned and '999' when service has not been provisioned successfully
resultDesc	String	No	'Success' when service has been successfully provisioned and 'Failure' when service has not been declined

transactionID	Long	Yes	Broker's serial unique identifier of the transaction
serviceID	Long	Yes	Third party's unique identifier of the transaction
serviceReceipt	String	Yes	Third party's unique identifier of the transaction
serviceDate	Date	Yes	Date when service was offered against the MPESA Payment by service provider [Format] yyyy-MM-dd HH:mm:ssZ [Example] 2014-10-26 09:29:54+0300
Initiator	String	No	Identifier who makes transaction on behalf of Third party that is defined in Mpesa
initiatorPassword	String	No	Base64 string of Initiator's password encrypted as defined below (Password Encryption)
conversationID	String	Yes	The unique identifier of the request message generated by broker/MPESA
originatorConversationID	String	Yes	The unique identifier of the request message generated by broker

7. Interface for C2B/B2B Payment Notification (Collection-only Account)

When payment is made into a Collection account from MPESA subscriber (C2B) or another business (B2B), MPESA Broker can be configured to notify the third party business system payment details instantly.

MPESA Core API Broker support several standard data exchange interfaces which include XML/HTTP, FTP, SMS and SOAP. Also it has a support for ISO8583 interface.

In all interfaces, Third party has to share the credentials (if applicable) and URL to access her system.

When XML/HTTP or FTP are used as exchange interface, Third party can choose to receive data either encrypted or plain format. The supported encryption algorithm is DES with CBC mode.

The data that can be shared includes transaction date, receipt number, reference number, transaction amount, language preference, customer name and MSISDN. For aggregators, specific Third party name can be included also in the data shared.

7.1. XML over HTTP

MPESA Core API Broker supports XML/HTTP data transfer through HTTP POST method. Third party has to specify the XML format supported by their system but should be limited to the data aforementioned.

If the format is not specified, the default one will be used.

```
<?xml version="1.0" encoding="UTF-8"?>
<transaction>
  <timestamp>2016/08/17 16:06:03+0300</timestamp>
  <service>Third party 01</service>
  <reference>0</reference>
  <receipt>30ZX987D</receipt>
  <amount>30000</amount>
  <msisdn>255764533524</msisdn>
  <name>FOO BAR</name>
</transaction>
```

Broker Request Attributes Definition

Attribute Name	Type	Optional	Description
timestamp	String	Yes	yyyy/MM/dd HH:mm:ss Z transaction time on MPESA system
service	String	No	Name used for Third Party account
reference	Long	Yes	Pay bill reference number
receipt	Long	Yes	MPESA receipt number
amount	String	No	Amount paid for the service
msisdn	String	No	Customer's phone number
Name	String	Yes	Customer's name

Broker shall be sending one transaction per post but in a multithreaded manner in case there are many transactions to push. The expected response from the Third Party is xml with 'Success' or 'Duplicate transaction' and a receipt number, else it shall be considered Failure.

Successfully received transaction notification request shall be responded as (for fresh transactions)

```
<?xml version="1.0" encoding="UTF-8" ?>
<response>
  <receipt>30ZX987D</receipt>
  <result>SUCCESS</result>
</response>
```

For duplicate transactions notification the third-party system shall respond as

```
<?xml version="1.0" encoding="UTF-8" ?>
<response>
  <receipt>30ZX987D</receipt>
  <result>DUPLICATE_TXN</result>
</response>
```

In both cases the Broker shall mark the transaction notification as successfully delivered to the third party system. This is irrespective of whether the transaction has been processed successfully (thereafter) or not. Thus the third party system can respond to the broker immediately after successfully receiving the request and is not required to process the request to completion before responding to the broker.

If the service provisioning to the subscriber is not successful, the third party may initiate reversal to the transaction independently of the broker interaction using MPESA web interface or using the reversal request shared in this document.

If the response is not received within 20seconds, the connection will timeout and the transaction will be reposted after 50seconds. These are system parameters not for specific Third party and can be changed at any time to improve performance.

7.2. File Transfer Protocol - FTP

MPESA Core API Broker can push CSV files to Third Party's FTP server with the header, **DATE, PHONE_NUMBER, AMOUNT, REFERENCE_NUMBER, RECEIPT_NUMBER, BILLER_CODE**

Date format will be **yyyy/MM/dd HH:mm:ss Z**.

The filename for the file transferred will be "datetime".csv i.e. "yyyyMMddHHmmss".csv

URL has to be shared as well as the path to put the files.

Below is File sent to FTP server

```
DATE, PHONE_NUMBER, AMOUNT, REFERENCE_NUMBER, RECEIPT_NUMBER, BILLER_CODE
2012/12/03 17:42:54 +0300, 255753245625, 1000.0, 53245625, L25IR020, 100300
```

2012/12/03 17:42:56 +0300, 255763445642, 1500.0, 25232562, L26I4020, 100300

2012/12/03 17:43:00 +0300, 255754436628, 18000.0, 11128175, L2IR9201, 100300

7.3. Simple Object Access Protocol - SOAP

SOAP is a protocol specification for exchanging structured information in the implementation of Web Services. MPESA Core API Broker can send pay bills transactions to

Web Service server implemented on the Third party side based on the WSDL included in this specification in Appendix.

Single transaction is sent per post and a Response Code, 200, on successful is expected in return.

Appendix

The appendix contains samples for different interfaces

C2B Notification Simple Object Access Protocol - SOAP Interface Specification

Below is the WSDL file to generate stubs for BI client to send data

```
<?xml version="1.0" encoding="UTF-8"?>
<wsdl:definitions
  name="BillService"
  targetNamespace="http://tz.co.infowise.mpesa.billers.middleware.model.bill"
  xmlns:impl="http://tz.co.infowise.mpesa.billers.middleware.model.bill"
  xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
  xmlns:wsdlsoap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema" >
  <wsdl:definitions
    name="BillService"
    targetNamespace="http://tz.co.infowise.mpesa.billers.middleware.model.bill"
    xmlns:impl="http://tz.co.infowise.mpesa.billers.middleware.model.bill"
    xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
    xmlns:wsdlsoap="http://schemas.xmlsoap.org/wsdl/soap/"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema" >
    <!-- WSDL Types -->
    <wsdl:types>
      <xsd:schema elementFormDefault="qualified"
        targetNamespace="http://tz.co.infowise.mpesa.billers.middleware.model.bill"
        xmlns:impl="http://tz.co.infowise.mpesa.billers.middleware.model.bill"
        xmlns="http://www.w3.org/2001/XMLSchema" >
        <xsd:element name="load">
          <xsd:annotation>
            <xsd:documentation>
              </xsd:documentation>
            </xsd:annotation>
          <xsd:complexType>
            <xsd:sequence>
              <xsd:element name="receiptNumber" type="xsd:string"
                nillable="false">
                <xsd:annotation>
```

```
<xsd:documentation>
</xsd:documentation>
</xsd:annotation>
</xsd:element>
</xsd:sequence>
</xsd:complexType>
</xsd:element>
<xsd:element name="loadResponse">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="loadReturn" type="impl:BillVO"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
<xsd:element name="postBill">
  <xsd:annotation>
    <xsd:documentation>
    </xsd:documentation>
  </xsd:annotation>
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="referenceNumber" type="xsd:string"
nillable="false">
        <xsd:annotation>
          <xsd:documentation>
          </xsd:documentation>
        </xsd:annotation>
      </xsd:element>
      <xsd:element name="amount" type="xsd:double"
nillable="false">
        <xsd:annotation>
          <xsd:documentation>
          </xsd:documentation>
        </xsd:annotation>
      </xsd:element>
      <xsd:element name="mpesaReceiptNumber"
type="xsd:string" nillable="false">
        <xsd:annotation>
```

```
<xsd:documentation>
</xsd:documentation>
</xsd:annotation>
</xsd:element>
<xsd:element name="phoneNumber" type="xsd:long"
nillable="false">
  <xsd:annotation>
    <xsd:documentation>
    </xsd:documentation>
  </xsd:annotation>
</xsd:element>
<xsd:element name="transactionDate"
type="xsd:dateTime" nillable="false">
  <xsd:annotation>
    <xsd:documentation>
    </xsd:documentation>
  </xsd:annotation>
</xsd:element>
</xsd:sequence>
</xsd:complexType>
</xsd:element>
<xsd:element name="postBillResponse">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="postBillReturn"
type="xsd:string"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
<xsd:element name="postBills">
  <xsd:annotation>
    <xsd:documentation>
    </xsd:documentation>
  </xsd:annotation>
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="businessNumber" type="xsd:long"
nillable="false">
```

```
<xsd:annotation>
  <xsd:documentation>
  </xsd:documentation>
</xsd:annotation>
</xsd:element>
<xsd:element name="date" type="xsd:dateTime"
nillable="false">
  <xsd:annotation>
    <xsd:documentation>
    </xsd:documentation>
  </xsd:annotation>
</xsd:element>
<xsd:element name="bills" type="impl:ArrayOfBillVO"
nillable="false">
  <xsd:annotation>
    <xsd:documentation>
    </xsd:documentation>
  </xsd:annotation>
</xsd:element>
</xsd:sequence>
</xsd:complexType>
</xsd:element>
<xsd:element name="postBillsResponse">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="postBillsReturn"
type="xsd:string"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
<xsd:complexType name="BillVO">
  <xsd:annotation>
    <xsd:documentation>
    </xsd:documentation>
  </xsd:annotation>
  <xsd:all>
    <xsd:element name="transactionDate" nillable="false"
type="xsd:dateTime">
```

```
<xsd:annotation>
  <xsd:documentation>
  </xsd:documentation>
</xsd:annotation>
</xsd:element>
<xsd:element name="amount" nillable="false"
type="xsd:double">
  <xsd:annotation>
    <xsd:documentation>
    </xsd:documentation>
  </xsd:annotation>
</xsd:element>
<xsd:element name="phoneNumber" nillable="false"
type="xsd:long">
  <xsd:annotation>
    <xsd:documentation>
    </xsd:documentation>
  </xsd:annotation>
</xsd:element>
<xsd:element name="referenceNumber" nillable="false"
type="xsd:string">
  <xsd:annotation>
    <xsd:documentation> </xsd:documentation>
  </xsd:annotation>
</xsd:element>
<xsd:element name="receiptNumber" nillable="false"
type="xsd:string">
  <xsd:annotation>
    <xsd:documentation>
    </xsd:documentation>
  </xsd:annotation>
</xsd:element>
<xsd:element name="id" nillable="false" type="xsd:long">
  <xsd:annotation>
    <xsd:documentation>
    </xsd:documentation>
  </xsd:annotation>
</xsd:element>
```

```
        </xsd:all>
    </xsd:complexType>
    <xsd:complexType name="ArrayOfBillVO">
        <xsd:sequence>
            <xsd:element name="billVO" minOccurs="0"
                maxOccurs="unbounded" form="qualified" type="impl:BillVO" />
        </xsd:sequence>
    </xsd:complexType>
</xsd:schema>
</wsdl:types>
<!-- Messages -->
<wsdl:message name="loadRequest">
    <wsdl:part element="impl:load" name="parameters"/>
</wsdl:message>
<wsdl:message name="loadResponse">
    <wsdl:part element="impl:loadResponse" name="parameters"/>
</wsdl:message>
<wsdl:message name="postBillRequest">
    <wsdl:part element="impl:postBill" name="parameters"/>
</wsdl:message>
<wsdl:message name="postBillResponse">
    <wsdl:part element="impl:postBillResponse" name="parameters"/>
</wsdl:message>
<wsdl:message name="postBillsRequest">
    <wsdl:part element="impl:postBills" name="parameters"/>
</wsdl:message>
<wsdl:message name="postBillsResponse">
    <wsdl:part element="impl:postBillsResponse" name="parameters"/>
</wsdl:message>
<!-- Port Type -->
<wsdl:portType name="BillServicePortType">
    <wsdl:operation name="load">
        <wsdl:documentation>
        </wsdl:documentation>
        <wsdl:input message="impl:loadRequest"
            name="loadRequest"/>
        <wsdl:output message="impl:loadResponse"
            name="loadResponse"/>
    </wsdl:operation>
</wsdl:portType>
```



```
</wsdl:operation>
<wsdl:operation name="postBill">
  <wsdl:documentation>
  </wsdl:documentation>
  <wsdl:input message="impl:postBillRequest"
    name="postBillRequest"/>
  <wsdl:output message="impl:postBillResponse"
    name="postBillResponse"/>
</wsdl:operation>
<wsdl:operation name="postBills">
  <wsdl:documentation>
  </wsdl:documentation>
  <wsdl:input message="impl:postBillsRequest"
    name="postBillsRequest"/>
  <wsdl:output message="impl:postBillsResponse"
    name="postBillsResponse"/>
</wsdl:operation>
</wsdl:portType>
<!-- WSDL Binding -->
<wsdl:binding name="BillServiceSoapBinding"
  type="impl:BillServicePortType">
  <wsdlsoap:binding style="document"
    transport="http://schemas.xmlsoap.org/soap/http"/>
  <wsdl:operation name="load">
    <wsdlsoap:operation soapAction="load" style="document"/>
    <wsdl:input name="loadRequest">
      <wsdlsoap:body use="literal"/>
    </wsdl:input>
    <wsdl:output name="loadResponse">
      <wsdlsoap:body use="literal"/>
    </wsdl:output>
  </wsdl:operation>
  <wsdl:operation name="postBill">
    <wsdlsoap:operation soapAction="postBill" style="document"/>
    <wsdl:input name="postBillRequest">
      <wsdlsoap:body use="literal"/>
    </wsdl:input>
    <wsdl:output name="postBillResponse">
```

```
<wsdlsoap:body use="literal"/>
</wsdl:output>
</wsdl:operation>
<wsdl:operation name="postBills">
  <wsdlsoap:operation soapAction="postBills" style="document"/>
  <wsdl:input name="postBillsRequest">
    <wsdlsoap:body use="literal"/>
  </wsdl:input>
  <wsdl:output name="postBillsResponse">
    <wsdlsoap:body use="literal"/>
  </wsdl:output>
</wsdl:operation>
</wsdl:binding>
<!-- Service Definition -->
<wsdl:service name="BillService">
  <wsdl:documentation>
  </wsdl:documentation>
  <wsdl:port binding="impl:BillServiceSoapBinding" name="BillService">
    <wsdlsoap:address
      location="http://localhost:8080/billerws/services/BillService"/>
    </wsdl:port>
  </wsdl:service>
</wsdl:definitions>
001/XMLSchema">
  <!-- WSDL Types -->
  <wsdl:types>
    <xsd:schema elementFormDefault="qualified"

targetNamespace="http://tz.co.infowise.mpesa.billers.middleware.model.bill"
    xmlns:impl="http://tz.co.infowise.mpesa.billers.middleware.model.bill"
    xmlns="http://www.w3.org/2001/XMLSchema">
    <xsd:element name="load">
      <xsd:annotation>
        <xsd:documentation>
        </xsd:documentation>
      </xsd:annotation>
      <xsd:complexType>
        <xsd:sequence>
```

```
<xsd:element name="receiptNumber" type="xsd:string"
nillable="false">
  <xsd:annotation>
    <xsd:documentation>
    </xsd:documentation>
  </xsd:annotation>
</xsd:element>
</xsd:sequence>
</xsd:complexType>
</xsd:element>
<xsd:element name="loadResponse">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="loadReturn" type="impl:BillVO"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
<xsd:element name="postBill">
  <xsd:annotation>
    <xsd:documentation>
    </xsd:documentation>
  </xsd:annotation>
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="referenceNumber" type="xsd:string"
nillable="false">
        <xsd:annotation>
          <xsd:documentation>
          </xsd:documentation>
        </xsd:annotation>
      </xsd:element>
      <xsd:element name="amount" type="xsd:double"
nillable="false">
        <xsd:annotation>
          <xsd:documentation>
          </xsd:documentation>
        </xsd:annotation>
      </xsd:element>
```

```
<xsd:element name="mpesaReceiptNumber"
type="xsd:string" nillable="false">
  <xsd:annotation>
    <xsd:documentation>
    </xsd:documentation>
  </xsd:annotation>
</xsd:element>
<xsd:element name="phoneNumber" type="xsd:long"
nillable="false">
  <xsd:annotation>
    <xsd:documentation>
    </xsd:documentation>
  </xsd:annotation>
</xsd:element>
<xsd:element name="transactionDate"
type="xsd:dateTime" nillable="false">
  <xsd:annotation>
    <xsd:documentation>
    </xsd:documentation>
  </xsd:annotation>
</xsd:element>
</xsd:sequence>
</xsd:complexType>
</xsd:element>
<xsd:element name="postBillResponse">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="postBillReturn"
type="xsd:string"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
<xsd:element name="postBills">
  <xsd:annotation>
    <xsd:documentation>
    </xsd:documentation>
  </xsd:annotation>
  <xsd:complexType>
```

```
<xsd:sequence>
  <xsd:element name="businessNumber" type="xsd:long"
nillable="false">
    <xsd:annotation>
      <xsd:documentation>
        </xsd:documentation>
      </xsd:annotation>
    </xsd:element>
    <xsd:element name="date" type="xsd:dateTime"
nillable="false">
      <xsd:annotation>
        <xsd:documentation>
          </xsd:documentation>
        </xsd:annotation>
      </xsd:element>
      <xsd:element name="bills" type="impl:ArrayOfBillV0"
nillable="false">
        <xsd:annotation>
          <xsd:documentation>
            </xsd:documentation>
          </xsd:annotation>
        </xsd:element>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
<xsd:element name="postBillsResponse">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="postBillsReturn"
type="xsd:string"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
<xsd:complexType name="BillV0">
  <xsd:annotation>
    <xsd:documentation>
      </xsd:documentation>
    </xsd:annotation>
```

```
<xsd:all>
  <xsd:element name="transactionDate" nillable="false"
type="xsd:dateTime">
    <xsd:annotation>
      <xsd:documentation>
      </xsd:documentation>
    </xsd:annotation>
  </xsd:element>
  <xsd:element name="amount" nillable="false"
type="xsd:double">
    <xsd:annotation>
      <xsd:documentation>
      </xsd:documentation>
    </xsd:annotation>
  </xsd:element>
  <xsd:element name="phoneNumber" nillable="false"
type="xsd:long">
    <xsd:annotation>
      <xsd:documentation>
      </xsd:documentation>
    </xsd:annotation>
  </xsd:element>
  <xsd:element name="referenceNumber" nillable="false"
type="xsd:string">
    <xsd:annotation>
      <xsd:documentation> </xsd:documentation>
    </xsd:annotation>
  </xsd:element>
  <xsd:element name="receiptNumber" nillable="false"
type="xsd:string">
    <xsd:annotation>
      <xsd:documentation>
      </xsd:documentation>
    </xsd:annotation>
  </xsd:element>
  <xsd:element name="id" nillable="false" type="xsd:long">
    <xsd:annotation>
      <xsd:documentation>
```

```
        </xsd:documentation>
    </xsd:annotation>
</xsd:element>
</xsd:all>
</xsd:complexType>
<xsd:complexType name="ArrayOfBillVO">
    <xsd:sequence>
        <xsd:element name="billVO" minOccurs="0"
            maxOccurs="unbounded" form="qualified" type="impl:BillVO" />
    </xsd:sequence>
</xsd:complexType>
</xsd:schema>
</wsdl:types>
<!-- Messages -->
<wsdl:message name="loadRequest">
    <wsdl:part element="impl:load" name="parameters"/>
</wsdl:message>
<wsdl:message name="loadResponse">
    <wsdl:part element="impl:loadResponse" name="parameters"/>
</wsdl:message>
<wsdl:message name="postBillRequest">
    <wsdl:part element="impl:postBill" name="parameters"/>
</wsdl:message>
<wsdl:message name="postBillResponse">
    <wsdl:part element="impl:postBillResponse" name="parameters"/>
</wsdl:message>
<wsdl:message name="postBillsRequest">
    <wsdl:part element="impl:postBills" name="parameters"/>
</wsdl:message>
<wsdl:message name="postBillsResponse">
    <wsdl:part element="impl:postBillsResponse" name="parameters"/>
</wsdl:message>
<!-- Port Type -->
<wsdl:portType name="BillServicePortType">
    <wsdl:operation name="load">
        <wsdl:documentation>
        </wsdl:documentation>
        <wsdl:input message="impl:loadRequest">
```

```
        name="loadRequest"/>
        <wsdl:output message="impl:loadResponse"
            name="loadResponse"/>
    </wsdl:operation>
    <wsdl:operation name="postBill">
        <wsdl:documentation>
        </wsdl:documentation>
        <wsdl:input message="impl:postBillRequest"
            name="postBillRequest"/>
        <wsdl:output message="impl:postBillResponse"
            name="postBillResponse"/>
    </wsdl:operation>
    <wsdl:operation name="postBills">
        <wsdl:documentation>
        </wsdl:documentation>
        <wsdl:input message="impl:postBillsRequest"
            name="postBillsRequest"/>
        <wsdl:output message="impl:postBillsResponse"
            name="postBillsResponse"/>
    </wsdl:operation>
</wsdl:portType>
<!-- WSDL Binding -->
<wsdl:binding name="BillServiceSoapBinding"
    type="impl:BillServicePortType">
    <wsdlsoap:binding style="document"
        transport="http://schemas.xmlsoap.org/soap/http"/>
    <wsdl:operation name="load">
        <wsdlsoap:operation soapAction="load" style="document"/>
        <wsdl:input name="loadRequest">
            <wsdlsoap:body use="literal"/>
        </wsdl:input>
        <wsdl:output name="loadResponse">
            <wsdlsoap:body use="literal"/>
        </wsdl:output>
    </wsdl:operation>
    <wsdl:operation name="postBill">
        <wsdlsoap:operation soapAction="postBill" style="document"/>
        <wsdl:input name="postBillRequest">
```



```
<wsdlsoap:body use="literal"/>
</wsdl:input>
<wsdl:output name="postBillResponse">
  <wsdlsoap:body use="literal"/>
</wsdl:output>
</wsdl:operation>
<wsdl:operation name="postBills">
  <wsdlsoap:operation soapAction="postBills" style="document"/>
  <wsdl:input name="postBillsRequest">
    <wsdlsoap:body use="literal"/>
  </wsdl:input>
  <wsdl:output name="postBillsResponse">
    <wsdlsoap:body use="literal"/>
  </wsdl:output>
</wsdl:operation>
</wsdl:binding>
<!-- Service Definition -->
<wsdl:service name="BillService">
  <wsdl:documentation>
  </wsdl:documentation>
  <wsdl:port binding="impl:BillServiceSoapBinding" name="BillService">
    <wsdlsoap:address
      location="http://localhost:8080/billerws/services/BillService"/>
    </wsdl:port>
  </wsdl:service>
</wsdl:definitions>
```